Projet 1 — `factorlab`

Un moteur d'algèbre linéaire dense en C++, et un modèle factoriel de rendements construit dessus.

L'idée

Tu écris `Matrix<T>` à la main — allocation, règle de cinq, opérateurs, vues, produit matriciel optimisé, décompositions. C'est la partie « apprentissage du langage » : tout ce que tu as vu dans les 12 exercices s'y retrouve, et rien d'autre.

Puis tu t'en sers pour faire tourner, sur des rendements d'actions réels, les deux modèles qui sont simultanément la base de l'apprentissage statistique et la base de la gestion de portefeuille :

- **la régression linéaire multiple** — le modèle supervisé le plus simple, et le modèle factoriel standard en finance (Fama-French, modèles de risque type Barra) ;
- **l'analyse en composantes principales** — la réduction de dimension la plus utilisée en ML, et la façon standard d'extraire les facteurs de risque d'un univers d'actifs.

Aucune bibliothèque externe. Pas d'Eigen, pas de BLAS, pas de `<random>` sophistiqué : seulement la bibliothèque standard. Le but n'est pas d'écrire du code que tu utiliserais en production — c'est d'avoir écrit toi-même ce que tu appelles habituellement en une ligne de NumPy, et de pouvoir en parler.

Ce que tu pourras dire en entretien : *« J'ai écrit ma propre bibliothèque de matrices en C++, avec gestion manuelle de la mémoire et un GEMM par blocs qui atteint X % de la performance d'OpenBLAS, et je m'en suis servi pour estimer un modèle factoriel sur 5 ans de rendements quotidiens de 30 actions. »* C'est une phrase que tu peux défendre 20 minutes.

Arborescence cible

```
factorlab/
  CMakeLists.txt
  README.md
  include/factorlab/
      Matrix.hpp          # étapes 1-3, 6
      MatrixView.hpp      # étape 5
      gemm.hpp            # étape 4
      decompositions.hpp  # étape 7
      data.hpp            # étape 8
      factors.hpp         # étapes 8-9
```

```
src/
    data.cpp
tests/
    test_matrix.cpp
    test_decompositions.cpp
    test_factors.cpp
bench/
    bench_gemm.cpp
data/
    prices.csv          # produit par le script Python
scripts/
    fetch_data.py       # yfinance -> CSV
    reference.py        # NumPy -> valeurs de référence pour les tests
build/                   # jamais commité
```

Règle de discipline : à la fin de chaque étape, le projet **compile sans avertissement**, les tests passent, et tu fais un commit. Pas d'étape suivante avant que celle-ci ne soit verte. C'est aussi comme ça que ton historique Git racontera quelque chose à quelqu'un qui l'ouvre.

Partie I — Le moteur

Étape 0 — Squelette et données

À faire

1. Monte l'arborescence, le `CMakeLists.txt` (modèle de l'exercice 11) avec deux configurations : `build-debug` (sanitizers) et `build-release` (`-O2 -DNDEBUG -march=native`).
2. Recopie ton micro-framework de test (`VERIFIE`) dans `tests/`.
3. Écris `scripts/fetch_data.py` : télécharge via `yfinance` les cours de clôture ajustés quotidiens de ~30 actions du S&P 500 sur 5 ans, et écris `data/prices.csv` au format `date,AAPL,MSFT,...`. Prends des secteurs variés (technologie, énergie, banques, santé, consommation) : l'ACP de l'étape 9 sera beaucoup plus lisible.
4. Ajoute `build/` et `data/*.csv` au `.gitignore`, mais **commite le script**.

Questions - Pourquoi les cours *ajustés* et pas les cours bruts ? Que se passe-t-il sur ta série le jour d'un *split* si tu utilises les cours bruts ? - Y a-t-il des valeurs manquantes dans ton CSV (jours fériés décalés, titres introduits en cours de période) ? Comment comptes-tu les traiter, et quel biais introduit chaque choix ?

Étape 1 — La classe `Matrix`

À faire

Une matrice de `double` en **row-major**, stockée dans un seul bloc contigu.

```
class Matrix {
public:
    Matrix(std::size_t lignes, std::size_t colonnes);              // rempli de 0
    Matrix(std::size_t lignes, std::size_t colonnes, double v);
    Matrix(std::initializer_list<std::initializer_list<double>>);  // pour les tests

    static Matrix identite(std::size_t n);
    static Matrix depuisColonnes(const std::vector<std::vector<double>>&);

    std::size_t lignes()   const noexcept;
    std::size_t colonnes() const noexcept;

    double&       operator()(std::size_t i, std::size_t j);        // assert
    const double& operator()(std::size_t i, std::size_t j) const;
    double&       at(std::size_t i, std::size_t j);                // throw

    const double* donnees() const noexcept;                        // pour le GEMM

private:
    std::size_t lignes_, colonnes_;
    double*     data_;          // ÉTAPE 1 : pointeur brut, volontairement
};
```

Écris `operator<<` pour l'affichage, et une fonction `transpose()`.

Questions intermédiaires - Pourquoi row-major plutôt que column-major ? (Fortran, LAPACK et Eigen utilisent column-major par défaut ; NumPy et le C utilisent row-major.) Quelle conséquence ça a sur l'ordre des boucles à l'étape 4 ? - `operator()(i,j)` plutôt que `operator[][]` : pourquoi le second est-il quasiment impossible à faire proprement ? - Quel est le coût mémoire *réel* d'une `Matrix(4096, 4096)` ? Compare avec `sizeof(Matrix)`. Explique la différence à voix haute. - Ton `initializer_list` doit vérifier que toutes les lignes ont la même longueur. `assert` ou exception ? Justifie avec le critère de l'exercice 10.

Validation : `test_matrix.cpp` construit, lit, écrit, transpose, et vérifie qu'une matrice `const` est lisible mais pas modifiable (mets la ligne fautive en commentaire avec l'erreur du compilateur en note — ça prouve que tu as testé).

Étape 2 — Règle de cinq, et la mesurer

À faire

Écris les cinq fonctions spéciales **à la main** (pointeur brut, comme à l'exercice 6), puis dans `bench/` un programme qui compare, sur une matrice 2048×2048 :

Opération	Temps attendu
`Matrix b = a;`	~ ms (32 Mo copiés)
`Matrix b = std::move(a);`	~ ns
`std::vector<Matrix>` : 20 `push_back` sans `reserve`	?
idem avec `reserve(20)`	?
idem après avoir retiré `noexcept` du constructeur de déplacement	?

Questions intermédiaires - Mesure le rapport copie/déplacement. Il devrait dépasser 10 . Explique d'où vient ce facteur. - Retire le `noexcept` et refais la ligne 3. Que se passe-t-il ? **Note le chiffre**, c'est l'anecdote la plus efficace que tu pourras raconter sur ce projet. - Ta `Matrix` est-elle copiable *par accident* quelque part dans ton code ? Instrumente temporairement le constructeur de copie avec un compteur global et fais tourner tes tests : combien de copies involontaires trouves-tu ?

Validation : ASan ne signale ni fuite ni double-free sur toute la suite de tests.

Étape 3 — Arithmétique

À faire

`operator+`, `operator-`, `operator+=`, `operator-=`, `operator*` par un scalaire des deux côtés, `operator==` avec tolérance (`approxEgal(a, b, tol)` en fonction libre — on ne compare pas des `double` avec `==`), négation unaire.

Toute incompatibilité de dimensions lève une exception dédiée `ErreurDimension` portant les quatre dimensions en cause (exercice 10.2).

Questions intermédiaires - Applique le patron « `+=` membre, `+` libre prenant le premier argument par valeur » de l'exercice 2. Vérifie avec un compteur que `a + b + c + d` ne fait **qu'une seule** allocation au lieu de trois. Explique pourquoi. - Pourquoi `operator==` sur des flottants est-il un piège ? Ta tolérance doit-elle être absolue, relative, ou les deux ? Que fait `numpy.allclose` ? - (Optionnel, à ne faire que si le reste est fini.) `a + b + c` alloue quand même un temporaire. Les *expression templates* éliminent ça. Lis-en une description, comprends le principe — et **n'implémente pas** : le rapport complexité/bénéfice est mauvais ici, mais savoir de quoi il s'agit est un bon point en entretien.

Étape 4 — Le produit matriciel : là où tout se joue

C'est l'étape la plus importante du projet. Elle produit le chiffre que tu mettras sur ton CV, et c'est elle que tu réutiliseras au projet 3.

À faire — quatre versions successives, chacune mesurée

1. **Naïve `i-j-k`** : les trois boucles dans l'ordre naturel.
2. **Réordonnée `i-k-j`** : mêmes opérations, ordre des boucles changé.
3. **Transposée** : transposer `B` d'abord, puis produit scalaire ligne × ligne.
4. **Par blocs** : découpage en tuiles de taille `BS`, avec `i-k-j` à l'intérieur.

Mesure chacune sur N = 256, 512, 1024, 2048 en `-O2 -march=native`, en GFLOP/s ($2 \cdot N^3$ opérations flottantes).

Questions intermédiaires - La version 2 ne change *rien* aux calculs effectués. Pourquoi est-elle plusieurs fois plus rapide ? Décris précisément ce que fait chaque boucle interne en termes de lignes de cache (reprends le raisonnement de l'exercice 12). - Pourquoi l'écart entre les versions 1 et 2 grandit-il avec N ? À partir de quel N la matrice cesse-t-elle de tenir en cache L2 sur ta machine (`lscpu`) ? Le seuil observé correspond-il ? - Fais varier `BS` de 8 à 256 pour N = 2048 et trace la courbe. Où est l'optimum ? Rapporte-le à la taille de ton cache L1 : combien de tuiles `BS × BS` de doubles faut-il y loger simultanément pour que le schéma fonctionne ? - Compare à `numpy.dot` sur la même machine et la même taille. Tu vas perdre, largement. **De combien ?** Et surtout : qu'est-ce qu'OpenBLAS fait que tu ne fais pas ? (Micro-noyaux écrits en assembleur, AVX/FMA explicites, *packing* mémoire, parallélisation multi-thread.) Savoir énoncer précisément ce qui te manque vaut mieux que d'avoir un chiffre plus élevé. - Vérifie avec `-fopt-info-vec` (GCC) si le compilateur vectorise ta boucle interne. Si non, pourquoi ? L'aliasing de pointeurs empêche souvent la vectorisation — `__restrict__` change-t-il quelque chose ?

Validation : les quatre versions donnent le même résultat à 1e-10 près sur des matrices aléatoires, comparées à une référence NumPy exportée par `scripts/reference.py`.

Livrable de l'étape : un tableau GFLOP/s × (version, N) dans le README. C'est le cœur quantitatif du projet.

Étape 5 — Vues non-propriétaires

À faire

```
class MatrixView {          // ne possède rien : pointe dans une Matrix existante
public:
    MatrixView(Matrix& m, std::size_t i0, std::size_t j0,
               std::size_t nl, std::size_t nc);
    double& operator()(std::size_t i, std::size_t j);
```

```
    // + ConstMatrixView pour la version lecture seule
private:
    double*     base_;
    std::size_t nl_, nc_;
    std::size_t stride_;    // = colonnes de la matrice d'origine
};
```

Ajoute `ligne(i)`, `colonne(j)`, `bloc(i0, j0, nl, nc)` sur `Matrix`.

C'est ici que le lien avec la finance devient concret : dans ta matrice de rendements `R` de taille `T × N`, une **colonne** est l'historique d'un titre, une **ligne** est une coupe transversale à une date, et un **bloc** est une fenêtre glissante d'estimation. Tu manipuleras les trois à la partie II.

Questions intermédiaires - Pourquoi le `stride_` est-il indispensable ? Dessine sur papier où tombent les éléments d'une sous-matrice 2×2 extraite d'une matrice 5×5 en row-major. - Une vue est un **pointeur brut non-propriétaire** — exactement le cas légitime de l'exercice 7. Quelle est la règle de durée de vie que l'utilisateur doit respecter ? Que se passe-t-il si la `Matrix` source est détruite, ou simplement redimensionnée, pendant que la vue existe ? Écris ce cas, fais-le tourner sous ASan, et **note le message** — c'est le talon d'Achille de toute API à base de vues, y compris `std::span` et `std::string_view`. - Faut-il que `MatrixView` soit copiable ? Déplaçable ? Combien des cinq fonctions spéciales dois-tu écrire ? Justifie en une phrase. - Compare avec `std::span` (C++20) et `std::mdspan` (C++23). Qu'est-ce que la norme résout, et pourquoi ça a mis si longtemps ?

Étape 6 — Généricité

À faire

Passe à `template <typename T> class Matrix`, contraint par un concept `std::floating_point`. Fais fonctionner `Matrix<double>` et `Matrix<float>`.

Questions intermédiaires - Quels fichiers dois-tu réorganiser, et pourquoi ? (Exercice 8.) - Refais le benchmark GEMM en `float` : le rapport de temps avec `double` est-il de 2 ? Plus ? Moins ? Explique en termes de bande passante mémoire **et** de largeur des registres SIMD. - Sur ta matrice de covariance de l'étape 8, calcule le portefeuille à variance minimale en `float` puis en `double`. Les poids diffèrent-ils ? De combien ? Que t'apprend ce résultat sur le **conditionnement** d'une matrice de covariance d'actions — dont on sait qu'elles sont fortement corrélées ? C'est une vraie question de recherche quant, et une excellente réponse à donner en entretien. - Ajoute `Matrix<long double>` : combien de lignes as-tu dû écrire ? C'est le test qui dit si ta généricité est réelle ou cosmétique.

Étape 7 — Décompositions

À faire

Dans `decompositions.hpp` :

1. `resolutionTriangulaireInf/Sup` — substitution avant et arrière.
2. **Cholesky** : `A = L L` pour `A` symétrique définie positive. Lève si un pivot est 0.
3. `resoudreSDP(A, b)` — résolution de `A x = b` par Cholesky.
4. **QR par Householder** (ou Gram-Schmidt modifié si tu préfères commencer plus simple), et `resoudreMoindresCarres(A, b)` par QR.
5. `inverse(A)` via Cholesky, en résolvant `A X = I` colonne par colonne.

Questions intermédiaires - Complexité de Cholesky ? Combien d'opérations en moins qu'une LU générale, et pourquoi ? (Réponse attendue : $\sim n^3/3$ contre $\sim 2n^3/3$, symétrie.) - Que signifie l'échec de Cholesky sur une matrice de covariance empirique ? Que se passe-t-il quand tu as **plus d'actifs que d'observations** ($T < N$) ? Construis volontairement ce cas avec ton CSV et observe. C'est le problème central de l'estimation de covariance en grande dimension, et il mène directement au *shrinkage* de Ledoit-Wolf — que tu peux mentionner en entretien même sans l'implémenter. - Compare la résolution des moindres carrés par **équations normales** (`A A x = A b` via Cholesky) et par **QR**. Fabrique une matrice mal conditionnée (deux colonnes presque colinéaires — deux actions du même secteur, par exemple) et compare les solutions à la référence NumPy. Laquelle se dégrade en premier, et pourquoi le nombre de conditionnement est-il **élevé au carré** dans les équations normales ? - Pourquoi n'inverse-t-on presque jamais une matrice en pratique, préférant résoudre le système ? (Coût, stabilité numérique.) Question classique d'entretien.

Validation : chaque décomposition vérifiée contre `numpy.linalg` à 1e-10 sur des matrices aléatoires, et `L @ L.T == A` reconstruit.

Partie II — Le modèle factoriel

À partir d'ici, tu n'écris presque plus de C++ « nouveau » : tu utilises ce que tu as construit. C'est fait exprès — c'est le test qui dit si ton API est utilisable.

Étape 8 — Des prix aux facteurs

À faire

1. `data.cpp` : un lecteur de CSV (`std::ifstream`, `std::getline`, `std::from_chars` ou `std::stod`), renvoyant les noms de colonnes et une `Matrix<double>` de prix `T × N`.
2. Rendements logarithmiques : `R(t,i) = ln(P(t+1,i) / P(t,i))`, matrice `(T-1) × N`.

3. Statistiques descriptives par colonne : moyenne, écart-type annualisé (`× sqrt(252)`), asymétrie, kurtosis.

4. Matrice de covariance `Σ = (1/(T-1)) · R_centré · R_centré`, et matrice de corrélation.

5. **Régression multi-facteurs** : construis une matrice de facteurs `F` de taille `T × k` — au minimum le rendement du marché (moyenne équipondérée de ton univers), et si tu veux un facteur momentum et un facteur de volatilité que tu construis toi-même. Pour chaque titre `i`, estime par moindres carrés :

   `r_i = _i + _i f + ... + _ik f_k + _i`

   Sors, pour chaque titre : les , l' , le R^2, et l'écart-type résiduel.

6. **Ridge** : = (F F + I) ^{1} F r. Trace ( ) pour variant sur une échelle logarithmique.

Questions intermédiaires - Rendements arithmétiques ou logarithmiques ? Lesquels s'additionnent dans le temps, lesquels s'additionnent en coupe transversale (dans un portefeuille) ? Le fait que les deux ne puissent pas être vrais simultanément est un point de détail qui revient souvent en entretien. - Combien vaut ton R^2 moyen sur une régression à un seul facteur (le marché) ? Il devrait être élevé — de l'ordre de 0.3 à 0.6 sur des actions américaines liquides. S'il est proche de 0, tu as un bug ; s'il est proche de 1, tu as probablement régressé quelque chose sur lui-même (ton facteur marché contient le titre `i`). Comment corriger ce second problème ? - L' estimé est-il statistiquement significatif ? Calcule son erreur-type (_ · sqrt((F F) ^{1})) et le t-statistique. Combien de tes 30 titres ont un $|t| > 2$? Compare à ce qu'on attendrait **par pur hasard** avec 30 tests à 5 %. C'est le problème des tests multiples, et c'est le premier réflexe qu'on attend d'un chercheur quantitatif. - Que fait la ridge quand $\to 0$? Quand $\to \infty$? Pourquoi la régularisation aide-t-elle quand tes facteurs sont corrélés entre eux ? Fais le lien avec le conditionnement de F F vu à l'étape 7 — c'est **exactement le même problème numérique**, vu depuis la statistique.

Validation : `scripts/reference.py` recalcule tout avec NumPy/statsmodels ; tes sorties C++ coïncident à 1e-8.

Étape 9 — ACP et portefeuille à variance minimale

À faire

1. **ACP** de la matrice de corrélation par **itération de puissance avec déflation** :
   - itération de puissance pour le vecteur propre dominant ;
   - déflation Σ ← Σ − v v ;

- répéter pour obtenir les 5 premières composantes. Sors les valeurs propres, la part de variance expliquée, et les *loadings*.

2. **Portefeuille à variance minimale** sous contrainte `Σ w = 1` :

   `w = Σ⁻¹ 1 / (1 Σ⁻¹ 1)`

   résolu par Cholesky (jamais par inversion explicite — étape 7).

3. Compare la volatilité *ex ante* de ce portefeuille à celle du portefeuille équipondéré. Puis calcule sa volatilité **réalisée hors échantillon** : estime `Σ` sur les 3 premières années, applique les poids sur la 4 .

Questions intermédiaires - Quelle part de variance capte ta première composante principale ? Regarde ses *loadings* : sont-ils tous du même signe et d'amplitude comparable ? Si oui, tu viens de retrouver le **facteur marché** sans jamais l'avoir défini. Explique pourquoi ça n'a rien d'un accident. - Regarde la deuxième et la troisième composante. Séparent-elles tes secteurs ? Fais le lien avec ce que tu sais de l'ACP en apprentissage statistique : c'est le même algorithme, sur la même matrice, avec une interprétation métier différente. - L'itération de puissance converge en `O(( / ) )`. Sur tes données, combien d'itérations pour 1e-10 sur la première composante ? Sur la cinquième ? Pourquoi est-ce beaucoup plus lent, et à quel moment la déflation accumule-t-elle trop d'erreur pour être fiable ? (C'est la raison pour laquelle on utilise Jacobi ou une QR itérée en pratique — implémenter Jacobi est un bon bonus si tu as le temps.) - Le portefeuille min-variance *ex ante* est-il aussi le meilleur *hors échantillon* ? En général **non**, et l'écart est spectaculaire. Pourquoi ? (Erreur d'estimation sur `Σ`, positions extrêmes et à levier, instabilité temporelle des corrélations.) Regarde tes poids : y a-t-il des positions courtes de -200 % ? Ajoute une contrainte `w 0` par une projection simple et refais la mesure. - Cette dernière question est la plus importante du projet. **Un candidat qui montre un backtest brillant sans discuter ses limites est immédiatement suspect ; un candidat qui montre un résultat médiocre et explique correctement pourquoi est crédible.** Écris cette discussion dans le README.

Étape 10 — Finition

À faire

- Suite de tests complète, `ctest` vert, y compris en configuration sanitizers.
- `bench/` avec les tableaux GFLOP/s de l'étape 4 et un graphique (généré en Python).
- README : le problème, l'architecture, les résultats chiffrés, les **limites**, et les instructions de compilation.
- Historique Git lisible : un commit par étape au minimum, messages en anglais.
- Passe `clang-tidy` et `cppcheck` une fois, et corrige ce qui est pertinent.

Le README est ce qu'un recruteur lit — souvent la seule chose. Structure conseillée : problème → ce qui est implémenté → résultats (deux tableaux et deux figures) → validation contre NumPy → limites connues → build. Une page. Pas trois.

Ce que ça donne sur le CV

Deux formulations, à adapter une fois que tu auras tes vrais chiffres :

> **Dense Linear Algebra Engine & Equity Factor Model** — *C++20* - Implemented a templated dense matrix library from scratch (manual memory management, full rule-of-five, non-owning views, Cholesky and Householder QR decompositions), with a cache-blocked GEMM reaching **XX GFLOP/s**, i.e. XX % of OpenBLAS on the same hardware. - Built an equity factor model on top of it: log-return construction, covariance estimation, multi-factor OLS and ridge regression, and PCA by power iteration on 5 years of daily returns for 30 S&P 500 constituents; first principal component recovers the market factor (XX % of variance explained). - Validated every numerical routine against NumPy to 1e-8; documented the out-of-sample degradation of the minimum-variance portfolio and its estimation-error causes.

Le troisième point compte autant que les deux premiers.

Les questions qu'on te posera sur ce projet

Prépare-les explicitement — c'est l'objet même du projet.

Sur le C++ 1. Ta `Matrix` gère la mémoire à la main. Montre-moi ton constructeur de déplacement. Pourquoi `noexcept` ? 2. Que se passe-t-il si le `new` échoue au milieu de ton `operator=` ? 3. Ta `MatrixView` ne possède rien. Comment empêches-tu un utilisateur de la conserver après destruction de la matrice source ? (Réponse honnête : tu ne peux pas, en C++. Sache dire pourquoi, et ce que ça implique.) 4. Pourquoi ton GEMM par blocs est-il plus rapide ? Quelle est la taille de bloc optimale sur ta machine et pourquoi celle-là ? 5. Pourquoi ne pas avoir utilisé Eigen ? (Réponse : parce que le but était d'apprendre. En production j'utiliserais Eigen ou BLAS — et sache dire ce qu'ils font de plus.)

Sur les mathématiques et la finance 6. Pourquoi résoudre les moindres carrés par QR plutôt que par les équations normales ? 7. Qu'est-ce que la première composante principale d'un panier d'actions, concrètement ? 8. Pourquoi une matrice de covariance empirique est-elle singulière quand $T < N$? Qu'est-ce qu'on fait dans ce cas ? 9. Ton portefeuille min-variance est mauvais hors

échantillon. Pourquoi, et comment tu le corrigerais ? 10. Tu as 30 alphas estimés, 3 sont significatifs à 5 %. Qu'en conclus-tu ?

Si tu sais répondre aux dix, la ligne « C++ » de ton CV est défendable, et tu as en prime un projet marchés financiers. C'était l'objectif.

Garde-fous

- **Ne pars pas dans les expression templates, le SIMD manuel ou le multithreading.** C'est le projet 3. Ici, le blocage cache suffit, et un projet fini vaut infiniment mieux qu'un projet ambitieux abandonné en octobre.
- **Écris les tests au fur et à mesure.** Une décomposition QR non testée est une décomposition fausse.
- **Compile en Debug avec sanitizers par défaut**, en Release seulement pour mesurer.
- Si une étape te bloque plus de 3 heures, passe à la suivante et laisse un `TODO`. L'ordre 1→10 est un ordre logique, pas une contrainte.

Quand tu attaques, envoie-moi ton code à chaque fin d'étape — je le relis comme le ferait un reviewer, sur les points qui comptent en entretien.